RehabSense Firmware

ESP32-S3 firmware design, built to PRD v1.4

CSE/EE 475 — Embedded Systems Capstone / EE 542 • Team 1 • Autumn 2026
Aidan Goodman • firmware architecture
Instructor: Prof. Shwetak N. Patel • TAs: Alex Ching, Guilin Hu
Firmware design v2.0 • October 22, 2026 • replaces Prototype v0.1 of 16 October

1 Summary

This document specifies the RehabSense firmware and the interface contract between the firmware and the Python host client. It replaces *Prototype v0.1* of 16 October in full, and it describes firmware that now exists: 7,701 lines of C in `main/`, built to PRD v1.4 and exercised by five test suites that all pass.

PRD v1.4 is normative. Where this document and the PRD describe the same behaviour, the PRD's description governs, and this document's job is to say how that behaviour is realised on an ESP32-S3 — which task owns it, which register, which constant, which frame.

Four decisions of 22 October supersede the PRD and are built as stated here:

1. **One EMG channel for the prototype.** A single MyoWare 2.0 on the biceps: two electrodes on the muscle belly about 20 mm apart, plus one reference on the bony elbow landmark — three electrodes in all.
2. **ERM for the full production cycle.** The LRA is dropped. Every reference to it is gone from the firmware and from this document.
3. **A soft click at 80% of the target angle** — PRD §11.2 left this open; it is now built as a seventh haptic pattern.
4. **Fatigue advises and does not stop the session** — the other open item in PRD §11.2, now settled the same way.

What the single channel costs. Co-contraction is a quarter of the PRD §5.5 score and needs an antagonist envelope, which one sensor on the biceps does not provide. The component is therefore weighted zero and the score normalises by the weights in force, so the three remaining components take a third each rather than the best achievable score being capped at 75. Calibration loses its triceps step, and the donning check loses its ability to say *the sleeve is on backwards* — it can still catch a dead or misplaced electrode, which §10.1 explains. Everything that needs a second channel is guarded on `EMG_N_CHANNELS` rather than deleted, so the custom PCB's two-channel front end is a change of one constant.

Repository state. `cse475/firmware`: an ESP-IDF project. Six new files, thirty modified. The host client is 11 Python files in `firmware/host`, updated in lockstep so that the two sides cannot disagree about a byte.

Status markers below: [built] built and exercised [partial] partly built [hardware] waiting on hardware.

2 What changed, and why it matters

2.1 PRD critical features

PRD §3.2 critical feature		State
Single-channel sEMG acquisition	•	Continuous DMA, 2 kSPS
Dual-IMU elbow angle	•	Madgwick per sensor, relative rotation about the calibrated flexion axis
On-device repetition detection	•	Gated on biceps activation; counted on the return
On-device grading, 0–100	•	Range, tempo and smoothness, equally weighted
Compensation detection	•	Shoulder at 20°, trunk as true common mode, momentum
Haptic feedback, three outcomes	•	Clean, flagged, set complete
Fault and rest cues	•	Both built; the fault cue plays before the mute
Non-overlapping cue playback	•	Single slot, priority, and fault preemption
Driver configuration self-test	•	Register read-back after every write
Host pattern test	•	Any of the seven patterns by id
BLE streaming and control	•	All twelve opcodes dispatched
Host application	•	`bleak` client, live plot, CSV log, MQTT
Calibration under 30 s	•	Five steps, ∼16 s of capture
Fatigue detection	•	Median-frequency regression; advises only
Battery operation, 2 h	○	Sensing built; endurance is a hardware measurement
Custom PCB	—	Hardware workstream

2.2 The 29 October demonstration

All seven items of PRD §3.1 now work. The two that did not are the reason for most of this revision.

#	Demonstration item	State
1	Sleeve, sensors, actuator, pod	Hardware workstream
2	Don, check, calibrate, run a set with a haptic per repetition	• Works
3	Shoulder elevation detected live, distinct haptic, lowered score	• Works
4	A passively swung arm rejected — no repetition starts	• **Now works**
5	A fatiguing set, median frequency falling, rest cue firing	• Works
6	Hold duration, repetition target, target angle changed live	• **Now works**
7	Host showing live traces, per-repetition scores, summary	• Works

2.3 The structural changes

Subject	Was (v0.1)	Is now
Repetition start	Posture alone	**Biceps activation** and the start threshold (§7.2)
Where it is counted	At the top hold	On the **return**, after a confirming hold at extension
Grading	Range 40, activation 25, smoothness 20, steadiness 15	Range, tempo, smoothness — equal weight; steadiness reported, not scored
Smoothness	Normalised jerk	**Velocity-peak count**: one push scores 100
Muscle-on threshold	Fixed 0.15 %MVC	Rest mean $+3\sigma$ on, $+2\sigma$ off, with confirmation windows
Shoulder threshold	15°	20°
Trunk lean	Inferred from upper-arm deviation	**Common-mode** rotation of both IMUs
Calibration	Four phases, ∼95 s	Five steps, ∼16 s, individually repeatable
Haptic patterns	Five, two of them invented	Seven, per PRD §5.6 plus the soft click
Set completion	Only on host `STOP`	At `rep_target`, with the compound cue
Host configuration	None	Fourteen TLV parameters (§12.4)
Opcodes dispatched	8 of 12	**12 of 12**

3 Hardware configuration and pin map

IMUs on SPI2, EMG on ADC1, and one short I^2C link to the DRV2605L that carries traffic only at boot. No sensor data crosses a shared bus.

Signal	Pin	Peripheral	Note
EMG (biceps) RAW	GPIO4	ADC1_CH3	12 dB attenuation, ∼0–3.1 V window
EMG B, Phase 2	*GPIO5*	*ADC1_CH4*	*Unused on the prototype*
Battery sense	GPIO7	ADC1_CH6	2:1 divider, read while continuous mode is idle
SPI SCK / MOSI / MISO	GPIO12 / 11 / 13	SPI2 (FSPI)	10 MHz, shared by both IMUs
CS, upper / forearm IMU	GPIO10 / 14	GPIO out	
I^2C SDA / SCL	GPIO8 / 9	I2C0	DRV2605L only, 400 kHz
Haptic PWM	GPIO6	LEDC	DRV2605L IN/TRIG
Haptic enable	GPIO15	GPIO out	Hard mute, bus-independent
Status LED	GPIO38	RMT	On-board WS2812, DevKitC-1 v1.1
User button	GPIO0	GPIO in	On-board BOOT button

Pins avoided: GPIO19–20 (USB D−/D+), GPIO26–32 (SPI flash), GPIO33–37 (octal PSRAM on the N8R8 module).

[built] **Electrode layout.** Three electrodes: two on the biceps belly about 20 mm apart and aligned with the muscle fibres, per SENIAM guidance, and one reference on the bony landmark at the elbow. The custom PCB adds the triceps pair and keeps the single shared reference, giving five.

[built] **Build configuration.** `sdkconfig.defaults` sets `CONFIG_SPIRAM_MODE_OCT=y` — the single most common cause of an N8R8 board reporting zero PSRAM — along with `CONFIG_FREERTOS_HZ=1000`, NimBLE only with a 247-byte preferred MTU, Wi-Fi disabled, and a 5-second task watchdog. The 1 kHz tick is what makes the 70–80 ms haptic gaps expressible; at the 100 Hz default they could not be timed at all.

[built] **BMI270 board configuration.** ADR jumper fully open (this is what selects SPI, not cut to GND and not left at default), I^2C pull-up jumpers cut, header pins mapped SCL→SCK, SDA→PICO, ADR→POCI, CS→CS. The device powers up in I^2C mode regardless; the driver performs a dummy read of register `0x00` and lets CSB rise, and that edge latches SPI mode.

3.1 Sampling rates

Stream	Rate	Note
EMG	2 kSPS	1 channel; MyoWare RAW carries content to ∼500 Hz
IMU	100 Hz per sensor	Accelerometer and gyroscope, both IMUs
Analysis	50 Hz	Hold timers and compensation confirmation need this
Feature frames on air	20 Hz	All a live plot requires [PRD §12]

[built] The two feature rates are now separate: analysis at 50 Hz, transmission decimated to 20 Hz. The saved bandwidth belongs to the raw stream, which is busiest exactly when tuning sessions need it.

4 EMG acquisition and signal chain

The acquisition layer is written against an abstract interface, so the prototype runs on the ESP32's internal ADC and the custom PCB runs on the ADS131M02 with no change above the driver. [built] Both backends are declared and the switch is a single assignment at initialisation; the ADS131M02 implementation is a stub that fails loudly, which is correct for a board that does not exist yet.

[built] **Internal ADC configuration.** Continuous DMA mode, not one-shot. One-shot sampling from a timer callback produces jitter that smears the spectrum, which is exactly the measurement fatigue detection depends on — PRD §5.8 makes this a load-bearing choice. `adc_continuous` on ADC1, `ADC_ATTEN_DB_12`, curve-fitting calibration, and a conversion-done callback that fills 64 samples and posts to the DSP task by queue.

4.1 [built] DSP chain

Run in order on the raw stream at 2 kSPS, using ESP-DSP (`dsps_biquad_f32`, `dsps_fft2r_fc32`).

1. **High-pass, 20 Hz, 2nd-order Butterworth.** Electrode half-cell drift and motion artefact, and incidentally the internal ADC's offset error.
2. **Low-pass, 450 Hz, 2nd-order Butterworth.**
3. **Notch, 60 Hz, $Q \approx 30$.** A 180 Hz harmonic notch sits behind a compile-time flag.
4. **Clip detection.** Samples within 1% of either rail, reported in telemetry so front-end gain is set against a counter rather than by eye.

5. **Rectify and smooth.** Moving RMS over 150 ms, decimated to 50 Hz.
6. **Normalise.** Zeroed below the rest floor, then scaled so the calibrated maximum reads 1.0.
7. **Median frequency.** 512-sample Hann-windowed FFT with 50% overlap, computed only while the envelope shows real activation.

[tested] **Verified numerically.** `test/verify_dsp.py` measures the assembled chain against known signals: −3 dB at both corners (0.707 at 20 Hz and 450 Hz), 0.000 gain at 60 Hz, 0.998 at 100 Hz. Median frequency lands within 2.1 Hz on pure tones at 80, 150 and 300 Hz, and falls monotonically as a synthetic spectrum shifts downward — the fatigue signature the whole feature depends on.

[built] **Mains measurement, for free.** The contact check needs to know how much 60 Hz the leads are picking up. Rather than a second spectral estimate, the chain taps either side of the notch: the power the notch removes *is* the mains pickup, because the notch is narrow enough to take essentially nothing else. Two slow exponential averages and a ratio.

4.2 [built] Muscle activation detector

PRD §5.4 specifies the detector that gates repetition start, and it is not a fixed threshold.

Transition	Condition	Confirmation
Activation on	Envelope above rest mean $+3\sigma$	Sustained 50 ms
Activation off	Envelope below rest mean $+2\sigma$	Sustained 100 ms

Rest mean and σ both come from calibration step 1, so the detector adapts to each session's own noise floor. It lives in `main/emg/activation.c`, its own translation unit, so the host test build can link it without the biquad chain and the FFT.

The hysteresis is not decoration, and neither is the unit conversion.

A single-threshold detector chatters at the boundary, and chatter on this signal means repetitions starting and abandoning at random. The asymmetric windows — fast to assert, slow to release — are what make the gate usable on a real limb. [tested] The suite drives 100 ticks of deliberate chatter either side of the midpoint and sees zero transitions.

The subtler trap is that calibration measures in raw RMS while everything downstream sees a *normalised* envelope: zeroed below rest × 1.5, then scaled so the maximum reads 1.0. A threshold expressed in raw units that is not put through the same transform lands somewhere else entirely. `activation_init()` applies it, and both sides read the multiplier from one constant — if those diverge, the muscle-on gate silently stops meaning what calibration measured.

A rest capture so quiet that 3σ falls below the floor the DSP already zeroes would make the gate fire on any non-zero sample, which is the opposite of gating. The threshold is floored just above zero, and a capture with no usable σ at all falls back to a fixed fraction of MVC and says so through `using_fallback` — it never fails open, because failing open restores exactly

the behaviour this replaces.

4.3 [built] Fatigue detection

Median frequency is tracked as a linear regression across the set. Three consecutive repetitions with slope below −0.5 Hz per repetition raise the flag and fire the rest cue once. A consistency guard requires each of the last few repetitions to sit below the one before, so a single noisy estimate cannot fire a cue at a patient.

Fatigue advises; it does not stop the session. The cue fires, the event is reported, and what to do about it is the patient's and the therapist's decision. A device that ended a set on its own reading of a frequency trend would be overruling the person doing the exercise.

5 IMU and elbow angle

Elbow angle is the *relative* orientation of the two BMI270s, never either one's absolute heading. With no magnetometer, absolute yaw is unobservable and drifts without bound.

Parameter	Value	Reason
Accelerometer range	$\pm 4\,g$	Curls stay well under $2\,g$
Gyroscope range	±500 dps	Margin over fast arm swing
Output data rate	100 Hz, both	Far above the ∼10 Hz content of a curl
Power mode	Performance	Low-power duty cycling adds timing jitter
Madgwick β	0.08	Tuning knob between drift and responsiveness

[built] **Bring-up hazard.** The BMI270 ships inert and needs an ∼8 kB configuration blob uploaded over SPI before it produces data at all. A device that reads the correct chip ID but returns zeros is almost always a failed upload. The init routine verifies `INTERNAL_STATUS` = `0x01` and the self-test fails explicitly if it does not.

5.1 [built] From quaternions to a joint angle

1. At the comfortable-straight reference pose, capture and store both orientations: `q_upper0`, `q_fore0`.
2. Each sample, compute each sensor's orientation relative to its own reference.
3. The relative rotation between those two is the joint rotation, independent of how either sensor is mounted.
4. Extract the flexion component about the dominant rotation axis, which is itself identified during the ROM sweep as the principal axis of the relative rotation — so sleeve alignment is never assumed.

5.2 [built] Gyroscope bias

New in this revision. Calibration step 1 captures per-axis bias for both sensors, and `fusion_update()` subtracts it before fusing. An uncorrected bias of even a fraction of a degree per second integrates into a visible angle error over a set, and it is the one error the patient cannot see. Bias is accumulated inside `fusion.c`, because that is where the raw samples arrive; calibration only ever sees derived values.

5.3 [built] Drift management

- **Session re-zero.** The reference pose is recaptured at the start of every session, resetting accumulated error to zero.
- **Rest-phase re-anchor.** When both IMUs show angular velocity below 3 dps and acceleration within 0.05 g of 1 g for more than 300 ms, the filter re-anchors to gravity. The confirming hold guarantees this once per repetition.

6 Compensation detection

All three compensations are observable from the two arm IMUs plus gravity, and all are evaluated only *during* a repetition. Compensation detected at any point is latched and reported at the confirming hold, so feedback arrives at one predictable moment rather than interrupting the movement.

Compensation	Signature	Threshold
Shoulder elevation	Upper-arm IMU rotated from its calibration pose	$> 20°$
Trunk lean	**Common-mode** rotation of both IMUs	$> 10°$
Arm swing	Upper-arm angular velocity, sustained	$> 30°/s$ for 200 ms
Sudden jolt	Acceleration spike on either IMU	$> 2.5\,g$

[built] **Trunk lean is now a measurement, not an inference — and that fixed a scoring bug.**

Trunk lean used to be inferred from a moderate upper-arm deviation while the elbow was nearly still. Because it triggered at 10° and shoulder elevation at 15°, and both tested the *same quantity*, any deviation past 15° with a slow elbow set both flags and cost −35 instead of −20. Every staged shoulder-compensation repetition was biased downward by an extra 15 points, and tuning grading weights against volunteer data would have absorbed the error rather than revealing it.

`fusion.c` now computes a true common-mode figure: the smaller of the two sensors' deviations, gated on their rotation axes pointing the same way. Trunk lean carries the whole arm, so both IMUs rotate together about nearly one axis; a shoulder hike moves the upper arm while the forearm follows the elbow instead, and the axes diverge. The two cannot collide.

[tested] Two cases pin it: a 30° upper-arm deviation with no common-mode component flags shoulder and *not* trunk; a 14° rotation shared by both sensors flags trunk.

Wrist substitution is not detectable with this sensor set. It needs a third IMU on the hand, which PRD §9 places out of scope.

7 Repetition detection

7.1 [built] The cycle

A repetition begins when the biceps activates and the arm rises past the start threshold, and is counted when the arm has returned below the finish threshold and been held still there for

`hold_duration_ms`.

Counting on the return makes the score cover the whole movement including the eccentric phase, where a weak limb usually fails first. The confirming hold supplies a stationary moment at a predictable point, which is what the haptic and the gravity re-anchor both need: feedback is scheduled, not reactive.

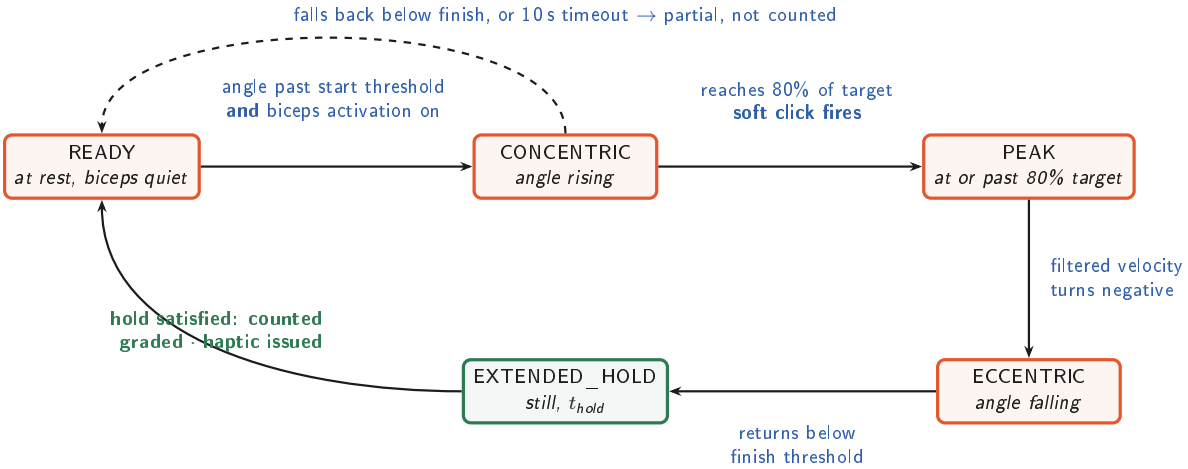


Figure 1: *The repetition cycle as built. Start is gated on biceps activation; the count happens after the arm returns to the extended position and holds still there.*

7.2 [built] The activation gate

This was the single largest gap in the firmware, and it is closed.

PRD §5.4 opens: "A repetition begins when the biceps activates *and* the arm rises past the start threshold." That conjunction is what allows a passively swung arm to be rejected — demonstration item 4, an acceptance criterion in §6, and the distinction on which the project's premise rests.

The old gate took `fusion_out_t` only. The EMG envelope was not a parameter and was not consulted anywhere on the path to a counted repetition; activation entered the system exactly once, where it was summed for the *score*. An arm swung passively through a full range, biceps silent throughout, satisfied every gate in turn and was **counted**.

`rep_cycle_tick()` now runs the activation detector on every tick and `READY` cannot be left without it. An arm that rises with the agonist quiet produces `REP_EVT_PASSIVE_SWING`: reported to the host, never counted, and it arms nothing.

[tested] The suite drives five full-range, correctly-postured, correctly-timed repetitions with the envelope held at zero. Nothing counts, nothing even starts, and the passive-swing event is raised. A separate case confirms the gate needs *both*: a static contraction at rest starts nothing either.

7.3 [built] Parameters

All of the following are host-configurable at runtime (§12.4) and take effect from the next repetition.

Parameter	Default	Basis
`target_angle`	110°	Therapist-set. Volunteers peaked at 89–98°; the therapist starts patients lower
"Target reached"	80% of target	The peak criterion, and where the soft click fires
`start_angle`	15°	20° for limited extension
`finish_angle`	10°	5° hysteresis below start
`hold_duration_ms`	1000, 300–3000	The primary tuning control: an impaired limb may not hold steadily for a full second
`hold_velocity_thresh`	15°/s	A hold requires the limb stationary
Direction reversal	5 Hz low-passed velocity < 0	A fixed rate jitters at the top of a slow curl
`tempo_min/max`	1.5 s, 3.5 s	Start to finish, excluding the hold
Repetition timeout	10 s → partial	The 21 Oct session contained 16.6 s and 24.7 s "repetitions"
`rep_target`	20, range 1–50	When the set-complete cue is issued

Thresholds are angles against the therapist's target, not fractions of measured ROM. The therapist prescribes what the patient *should* reach; measured ROM is what they currently can, and scoring against the latter would hand full marks to a shrinking range.

7.4 [built] Behavioural rules

Three rules hold regardless of parameters, and all three are implemented:

- Every completed repetition is counted and *then* graded. Nothing is silently discarded, because a counter that fails to advance gives the patient no interpretable information.
- A repetition flagged for compensation still counts toward the target. Quality lives in the score, not in the counter.
- Compensation detected at any point is latched and reported at the confirming hold.

[built] **The start-posture gate is separate from the activation gate.** A repetition begun with the upper arm already more than 15° off its calibration attitude is refused rather than counted and penalised: starting from a hike is a different error from developing one mid-curl, and the patient can fix it before they begin. [tested] Pinned by its own case.

8 Grading

Each repetition receives a score from 0 to 100. The patient receives only a pass/fail distinction, through the choice of haptic pattern; the number is shown in the application.

Component	Weight	Measure
Range of motion	1/3	Peak angle divided by the therapist's target angle
Tempo	1/3	Start threshold to finish threshold, against the window
Smoothness	1/3	Velocity peaks above 10% of this repetition's peak velocity; one push scores 100
Co-contraction	*0*	*Needs the second channel; Phase 2*
Deductions, applied after the mean; clamped to 0–100		
Shoulder compensation	−20	Upper-arm IMU past 20° from its calibration pose
Trunk lean	−15	Common-mode rotation of both IMUs past 10°
Momentum / arm swing	−15	Upper-arm angular velocity past 30°/s for 200 ms

Steadiness during the confirming hold is computed and *reported* but carries zero weight by default, per PRD §5.5. It is available as an optional fifth component by giving it a weight from the host.

[built] **The score normalises by the weights in force.** A host that switches a component off — or a build with one channel, where co-contraction has nothing to measure — redistributes that share across the rest rather than capping the best achievable score below 100. Without this, every patient on the prototype would score at most 75 for a perfect repetition, and the number on the therapist's screen would mean something different from the number in the specification. [tested] A perfect repetition scores 100 with tempo switched off.

8.1 [built] Smoothness: counting pushes

PRD §5.5 specifies a velocity-peak count rather than a jerk metric, and the reason is interpretability: "this movement had four separate pushes" means something to a therapist in a way that a normalised jerk figure does not.

Peaks are detected on a 5 Hz low-passed velocity so that sensor noise does not read as a correction the patient made, and their magnitudes are stored so the 10% threshold can be applied at the end, once the repetition's own peak is known.

Counting peaks is also naturally scale-free, which matters more than it sounds. [tested] An unnormalised jerk metric — the thing v0.1 shipped — handed a better score to anyone who simply moved slower, rewarding exactly the guarding behaviour this device exists to discourage. The suite pins the fix from both ends: a smooth curl at 2.4 s and the same shape at 3.2 s register the identical peak count, and a four-push curl over the same range registers more.

8.2 [built] Tempo

Full credit inside the prescribed window, falling off linearly over one window-width either side, so a repetition just outside is marked down rather than zeroed. Both directions matter: too fast is momentum, too slow is usually guarding or a stall mid-range.

Tempo is measured start threshold to finish threshold and *excludes* the confirming hold, which is why counting on the return matters — the whole excursion is inside the measurement.

9 Haptics

Feedback is delivered by an Adafruit 1201 ERM at the end of the confirming hold, when the patient is stationary.

9.1 [built] Drive path

An ERM's vibration frequency is coupled to its amplitude, so "stronger" also means "faster" and cannot carry independent meaning. Every pattern is therefore distinguished by **rhythm** — pulse count, width and gap — and the six result cues play at one consistent amplitude, which is also easier for a patient to learn than a set of intensity gradations.

Power-up sequence. Reset the DRV2605L; leave standby; select ERM drive (`N_ERM_LRA` = 0 in Feedback Control, `0x1A`); set the rated voltage and overdrive clamp; run auto-calibration; store the results in NVS; switch the mode register to PWM input; then read back every configured register. A mismatch fails the self-test before the session begins. If auto-calibration fails, the driver falls back to the stored calibration; if none exists, initialisation fails rather than running with unknown drive parameters.

[built] **PWM drive.** LEDC at 20 kHz, 10-bit, with 50% duty as zero drive and deviation from 50% setting amplitude. The output is held at zero drive before the chip is switched into PWM mode, so the actuator cannot twitch during configuration.

[built] **Onset.** Each pulse opens with a 20 ms overdrive kick. Without it an ERM's onset is soft and the pulse reads as a fade rather than a tap — the difference between a reward signal and a vague buzz. Active braking through the driver is what makes 60 ms gaps workable; without a driver IC the floor would be ∼120 ms, and the shortest gap any pattern asks for is 70 ms.

[hardware] **Actuator limits.** `ERM_RATED_VOLTAGE_V` = 3.0 and `ERM_OVERDRIVE_CLAMP_V` = 3.3 are starting values for a 2–5 V part. Confirm against the Adafruit 1201 datasheet during bring-up: wrong values here produce a weak or buzzy actuator.

9.2 [built] The seven patterns

Event	Pattern	Interpretation
Repetition counted, clean	One 100 ms click	Met the form, range and tempo criteria
Repetition counted, flagged	Three 80 ms pulses, 80 ms gaps	Counted, but incomplete range, outside the tempo window, or compensation
Final repetition of the set	Two 70 ms pulses with 70 ms gaps, then one 500 ms pulse	The repetition target has been reached
Rest now	One 1000 ms ramp up and down	Fatigue flag raised
Fault	Two 300 ms pulses, 300 ms apart	Contact lost, sensor fault, or low battery
Button press	One 40 ms tick	Confirms every press
80% of target	One 30 ms click at reduced level	**New:** you may begin the return

The soft click is the only cue that fires mid-movement, and it is shaped so that it cannot be confused with a result.

Everything else is delivered at a stationary moment and carries a verdict. The soft click is a waypoint, so it is the shortest and lightest in the set, it skips the overdrive kick that gives the others their snap, and it shares the button tick's priority — meaning it is *dropped* rather than displacing anything if a cue is already playing. A waypoint the patient misses costs nothing; a result they miss costs the repetition.

Two patterns from v0.1 are gone. `WRONG_START` played at 50% amplitude, which broke the single-amplitude rule, and nagged every 2.5 s while a patient was trying to get into position; the app now shows that instead. `BOOT_OK` was two 100 ms pulses 200 ms apart, sitting in the fault cue's rhythmic neighbourhood — the same class of collision the PRD records having had to fix once already. The status LED reports a successful boot.

[built] **Set completion.** On the repetition that reaches `rep_target`, a *compound* pattern plays the per-repetition result, a 300 ms gap, then the set-complete cue, so the patient receives both the quality result and confirmation that the set is finished. It is one request rather than two queued ones, because a single-slot scheduler would drop the second.

9.3 [built] Scheduling

Rule [PRD §5.6]	Implementation
One cue plays at a time	Queue of depth 1; the task plays to completion before taking another
A new cue takes the pending slot only on higher priority	`haptic_play()` peeks, compares, and resets only on a win
A fault preempts a playing cue and clears anything pending	A cancel flag checked between segments, and no delay runs longer than 10 ms without checking it
A tick is dropped if any cue is playing	Dropped on `haptic_is_busy()`
Priority: tick < result < set complete and rest < fault	As stated. Clean and flagged share a rank: they are one class, and exactly one is issued per repetition

[built] **Fault preemption is the rule that needed real work.** Pattern playback used to run to completion through `vTaskDelay` with nothing able to interrupt it. Delays are now sliced, so a fault reaches the actuator within about 10 ms however long the cue in flight was going to be.

[built] **The fault cue plays *before* the mute.** PRD §5.2: haptics are muted through the enable pin only after the fault cue finishes, "so the fault state cannot silence its own warning." The old `sm_fault()` muted first, which is right for a runaway motor and exactly wrong for a lifted electrode, where the cue *is* the information. The distinction is now which fault code was raised: `fault_warrants_cue()` in `rehab_types.h` names the faults the patient must be told about, and the mute is deferred 1 s for those and immediate for the rest. The deferral is a deadline checked in `sm_tick()`, not a sleep, so the analysis task is never blocked waiting for a pattern to finish.

9.4 Latency budget

Stage	Worst case
`analysis` task period	20 ms
Queue post and haptic task wake	< 2 ms
LEDC duty update	< 1 ms
ERM onset to perceptible, with overdrive	~20 ms
Total	**~43 ms**

Within the PRD's 50 ms budget, and measured from the completion of the confirming hold rather than from a movement.

[partial] **Timing exposure.** `CONFIG_FREERTOS_HZ=1000` gives 1 ms resolution, so the specified widths are expressible and the PRD's 10% tolerance is met on a quiet system. The exposure is preemption rather than resolution: the haptic task is pinned to core 0 alongside the NimBLE host, so a radio event during a pattern can stretch a gap. A scope capture during bring-up settles it; if it proves marginal, segment timing moves to a hardware timer and the task only arms it.

10 Calibration

Calibration runs as a sequence of short, separately commanded steps, so a failed step can be repeated on its own without restarting. [built] Capture totals about sixteen seconds on the single-channel prototype, against a Must of "under 30 s" in PRD §3.2 — down from roughly ninety-five.

10.1 [built] Donning check, step 0

1. **Both IMUs respond**, with elbow motion present during a prompted curl.
2. **The EMG channel shows life** — envelope above the noise floor.
3. **No saturation.** A clipping channel makes every other measurement untrustworthy, and clipping destroys the median-frequency estimate while leaving the envelope looking plausible.
4. **Mains pickup within limits**, measured as the power the 60 Hz notch removes.
5. **Placement**, below.

What one channel costs the donning check. The two-channel test is a comparison: during a prompted curl the biceps envelope must exceed the triceps, and if it does not, the sleeve is on backwards or the leads are crossed — and the device can say *which*.

With one sensor there is nothing to compare against, so that specific diagnosis is no longer available and the firmware does not pretend otherwise. What it checks instead is that the channel responds to the movement it is supposed to measure: the envelope while the elbow is moving must exceed its level while still by a clear factor. A pair on the muscle belly clears it comfortably; a lifted, misplaced or dead electrode does not.

That catches the common failures. It cannot distinguish "on the wrong muscle" from "on the right muscle but not contracting", and the message the patient sees says so rather than claiming more — it asks them to check the two electrodes and the reference, which is the

action either case needs.

10.2 [built] The steps

Step	Time	Patient does	Captured, and pass condition
0. Contact	4 s	Sits; sleeve on; one prompted curl	Saturation, mains, IMU response, placement
1. Rest	5 s	Arm relaxed at the side, completely still	EMG resting mean *and* σ, and *gyroscope bias*. Rejected if movement is detected
2. Straight arm	2 s	Straightens *as far as is comfortable* and holds still	The 0° reference pose and both neutral quaternions. Rejected if not still
3. Biceps maximum	3×3 s	Elbow at ~90°, pushes up against a table	100% reference, strongest 500 ms window, best of three
4. Triceps max	—	—	*Phase 2, with the second channel*
5. Range sweep	6 s	Two slow, full-range curls	Flexion axis as the principal axis of the relative rotation; θ_{min}, θ_{max}; baseline median frequency

10.3 [built] Four things the protocol now does

"Comfortable straight", not "hanging down". PRD §5.3 records the 21 October finding that step 2 must instruct *comfortable* straight: calibrating against a hyperextended pose caused returns to finish 10–12° short of the finish threshold, so repetitions did not complete. This is a prompt string and a pose, and it is the difference between a working session and one where nothing counts.

Stillness gates on steps 1 and 2. The phases used to accumulate unconditionally and never reject. Movement during the rest step corrupts gyroscope bias and therefore the joint angle for the whole session, with no obvious symptom — which is precisely why a gate is needed rather than a warning. A brief wobble is tolerated; sustained movement rejects the step with a message naming the cause.

σ is captured and kept. The old code accumulated `rest_sum_sq` faithfully for ten seconds and then never read it — only the mean reached the record. The σ the activation detector depends on was computed and thrown away. It is now a field in the calibration record, which is why the record is at version 2.

Steps are individually commanded. `CMD_START_CALIBRATION` has always carried a step argument on the wire, and the host has always tested re-running one step; the firmware discarded it. It is now honoured. The step most likely to fail is a maximum contraction on a paretic limb, which is also the one a patient least wants to repeat four others to get back to.

10.4 [built] Maximum contraction on an impaired limb

Maximal effort is inconsistent on a paretic limb, and a single poor attempt mis-scales every subsequent measurement. All three mitigations from PRD §5.3 are built: best of three attempts, each measured as the strongest 500 ms window rather than a single loudest sample; a plausibility floor that flags `MVC_UNRELIABLE` rather than silently accepting a maximum that barely clears the noise;

and a clinician-supplied value pushed from the host, written to the calibration characteristic, which sets `CALIB_FLAG_CLINICIAN_OVERRIDE` and clears the unreliable flag. That last path existed as a flag definition and nothing else.

10.5 [built] Persistence and the returning patient

Results are written to NVS under a patient profile key. Steps 1, 2 and 5 re-run every session, since they reset gyroscope bias and IMU drift and absorb day-to-day variation in sleeve placement; step 3 is reused from the stored profile until it expires.

[built] **The short path now actually runs.** Two one-line defects combined to prevent it: `calib_record_t.timestamp` was read by the staleness check and written nowhere, and `tick_donning()` passed a literal zero for the current epoch because the device had no wall clock. Every stored profile therefore read as expired, and every patient went through the full protocol every single session.

The record is now stamped at save time, the epoch comes from `CMD_SET_TIME`, and "no host time yet" is treated as *unknown age* rather than *expired* — the profile is still internally consistent, and forcing a full recalibration because the phone has not spoken yet is worse than using it.

11 Firmware state machine

Seven states. The host drives transitions, but the device refuses to enter `OPERATIONAL` until the donning check has passed and a calibration profile exists — the host cannot override that, because scoring against a bad reference is worse than not scoring at all.

State	BLE	Buffering	Exits on
`OFF`	Off	—	Power button
`BOOT / SELF_TEST`	Advertising	—	Self-test result
`STANDBY`	Connected, status	—	`START`, or short press
`DONNING_CHECK`	Status + envelope	—	Check pass or fail
`CALIBRATION`	Status + step progress	—	Protocol complete or abort
`OPERATIONAL`	Full stream	Tier 1 + tier 2	`STOP`, **set complete**, or fault
`FAULT`	Status only, cause	Sealed, readable	Reset

[built] **Set completion ends the set.** Reaching `rep_target` seals the buffer, issues the compound cue, emits `SET_COMPLETE` and returns to `STANDBY` for the rest interval. Previously a set ended only when the host sent `STOP`, and there was no `rep_target` anywhere in the firmware.

[built] **BLE dropout is not a state.** A device in `OPERATIONAL` with no connection stays in `OPERATIONAL`: acquisition, repetition detection, grading, haptics and tier-2 filling all continue, and only the transmit task idles. The publishers return `ESP_ERR_INVALID_STATE` when nobody is subscribed and every caller treats that as normal rather than as an error.

11.1 [built] Status indication

State	Colour	Pattern
Idle, advertising	Blue	Slow blink
Connected	Blue	Solid
Calibrating	Amber	Pulsing
Session running	Green	Fast blink
Session running, link down	Green	Magenta blink every 2 s
Battery below 15%	Amber	Double blink every 2 s
Fault	Red	Solid

[built] **Battery.** GPIO7 through a 2:1 divider. ADC1 runs in continuous DMA mode for EMG during a session and a one-shot read cannot share the unit, so the battery is sampled whenever acquisition is idle — at boot, in `STANDBY`, and between sets — and the last reading is held during a session. That is sufficient for the rule it serves: below 15% the device completes the current set, then stops and saves. The decision is taken at a set boundary, which is exactly when a fresh reading is available.

12 Buffering and data retention

12.1 The arithmetic

Stream	Rate	Bytes/s
EMG raw, 1 ch × 2 kSPS × int16	2 kHz	4,000
IMU, 2 × (3 accel + 3 gyro)× int16	100 Hz	2,400
Derived features and events	20 Hz	∼160
Total		**∼6.6 kB/s**

About 396 kB per minute on one channel, against 648 kB on two. The tiering is unchanged, and the margin is simply larger.

12.2 [built] Three tiers

Tier 0 — DMA landing buffers. Two 64-sample buffers ping-ponged by the ADC driver, about 32 ms each. Exists only to decouple the conversion callback from the DSP task.

Tier 1 — live ring in internal SRAM, 8 seconds. Feeds the DSP windows, serves the BLE transmit queue, and absorbs short connection stalls with no backfill machinery at all. Most real dropouts are sub-second and never escalate past this tier.

Tier 2 — session store in PSRAM, 600 seconds. Ten minutes of full-rate raw data. Reset on `START`, sealed on `STOP`. Allocated with `heap_caps_malloc(size, MALLOC_CAP_SPIRAM)` and verified at boot rather than assumed.

12.3 [built] Recovery after a dropout

Every frame carries a monotonic `uint32` sequence number that resets at `START`. The host acknowledges the highest contiguous sequence it has received; the device may release tier-2 storage below

that mark but holds everything above it. On reconnect the host reads the retained range from the status characteristic and requests what it missed.

[built] **The request half is now wired.** `CMD_REQUEST_BACKFILL` had an opcode, a host-side builder and a characteristic to notify on — and no `case` in the firmware's dispatch, so a host that asked for a replay got silence. This was a known gap in v0.1 and it closed the last PRD §6 acceptance criterion that could not be run at all.

A new `storage` task serves requests one chunk at a time, paced at a chunk every 20 ms, so catching up never stalls real-time data. If part of the requested range has aged out, the device serves what remains and sets the gap flag — it never silently serves a short range as though it were complete, because a host that trusts an incomplete range reconstructs a session that never happened.

13 BLE interface

This section is the boundary between the firmware and the host client. Everything is little-endian, and every sample carries a timestamp from a single monotonic device clock.

13.1 [built] Services and characteristics

Characteristic	Short UUID	Properties	Carries
Control	`A5B10002`	Write	Commands from host to device
Raw stream	`A5B10003`	Notify	EMG and IMU sample frames
Events and features	`A5B10004`	Notify	Feature frames and async events
Device status	`A5B10005`	Read, Notify	State, battery, signal quality, sequence range
Backfill	`A5B10006`	Notify	Replayed frames after a dropout
Calibration record	`A5B10007`	Read, Write	The `calib_record_t` struct
Configuration	`A5B10008`	Read, Notify	**New:** serialised TLV (§12.4)

13.2 [built] Frame formats

All stream frames share a 14-byte header: `type` (uint8), `seq` (uint32), `t_us` (uint64), `n` (uint8).

Type	Contents	Batch	Rate	Bytes
`0x01` EMG	$n\times$ int16 per channel	80 samples	25 frames/s	174
`0x02` IMU	$n \times (2 \times 6\times$ int16)	6 samples	16.7 frames/s	158
`0x03` Features	Envelope, MDF, angle, deviation, state, count, flags, activation	1	20 Hz	40
`0x04` Event	Event id plus payload	1	Async	≤ 32

[built] **The EMG batch derives from the channel count.** Each sample is 2 B per channel, so 80 samples of one channel and 40 of two both come to $14 + 160 = 174$ B — the frame is the same size whichever front end is fitted, and the frame *rate* falls out of it. A `_Static_assert` pins it inside the

180-byte budget, which exists because macOS caps ATT MTU at 185 and `bleak` does not expose a portable MTU request.

13.3 [built] Control commands

All twelve opcodes are now dispatched. Four were not: `SET_CONFIG`, `GET_CONFIG`, `SET_TIME` and `REQUEST_BACKFILL`.

Opcode	Command	Payload
0x01	START_SESSION	exercise id (uint8)
0x02	STOP_SESSION	—
0x03	START_CALIBRATION	step (uint8) — now honoured
0x04	ABORT_CALIBRATION	—
0x05	REQUEST_RECALIBRATION	—
0x06	ACK_SEQ	highest contiguous seq (uint32)
0x07	REQUEST_BACKFILL	start, end (uint32 × 2)
0x08	SET_CONFIG	TLV block (§12.4)
0x09	GET_CONFIG	—
0x0A	HAPTIC_TEST	pattern id (uint8)
0x0B	SET_TIME	host epoch microseconds (uint64)
0x0C	POWER_OFF	—

[built] **The config opcode is no longer overloaded.** A write to the *calibration* characteristic used to be forwarded to the command callback under `CMD_SET_CONFIG`. The two carry entirely different payloads, and the moment a real TLV parser existed, a `calib_record_t` would have been fed to it. Calibration writes now arrive under an internal opcode that never appears on the wire.

[built] **Time synchronisation.** On connection the host sends `SET_TIME` with its own epoch; the device records the *offset*, reports it in the status characteristic, and **never adjusts its own clock**. Adjusting a sampling clock to match a phone would corrupt exactly the timing the spectral analysis depends on — the fatigue index is a frequency measurement, and a frequency measured against a clock that jumps is not a measurement.

13.4 [built] Runtime configuration

PRD §5.9.1 requires every parameter below to be settable over BLE, changeable during a session, effective from the next repetition, stored in NVS and read back for confirmation. None of it existed; all of it does now, including the tag vocabulary, which neither side had defined.

Tag	Parameter	Type	Default and range
0x01	hold_duration_ms	uint16	1000, 300–3000
0x02	hold_velocity_thresh	float	15°/s
0x03	rep_target	uint8	20, 1–50
0x04	set_count	uint8	3
0x05	rest_seconds	uint16	60
0x06	target_angle	float	110°
0x07	start_angle	float	15°
0x08	finish_angle	float	10°
0x09	tempo_min, max	float × 2	1.5 s, 3.5 s
0x0A	Compensation thresholds	float × 3	20°, 10°, 30°/s
0x0B	Grading weights	float × 5	range, tempo, smoothness, co-contraction, steadiness
0x0C	Deductions	float × 3	20, 15, 15
0x0D	raw_stream_enable	uint8	On during tuning
0x0E	Exercise id, agonist channel	uint8 × 2	0, 0

Three properties of the TLV encoding earn their place.

Unknown tags are skipped; wrong lengths are rejected. A newer host may send tags an older build has never heard of, and skipping lets the rest of the block land. But a *recognised* tag with the wrong length means the two sides disagree about what it means, and guessing would write a plausible wrong value into a live session.

Every value is clamped to a range the device can honour. A configuration that cannot produce a countable repetition is worse than a rejected write, because it shows up as a device that has quietly stopped counting rather than as an error. The finish threshold is forced below the start threshold, since without hysteresis the cycle chatters between `READY` and `CONCENTRIC`; the peak criterion is forced reachable; and an all-zero weight vector restores the defaults rather than scoring every repetition zero.

`GET_CONFIG` returns the same block. That is how the host confirms a write took effect, and it is why the configuration has its own readable characteristic rather than being inferred from behaviour.

14 Task architecture and timing

[built] The NimBLE host owns core 0. Everything latency-sensitive goes on core 1, where radio activity cannot preempt it.

Task	Core	Prio	Trigger	Stack	Job
ADC conv-done	1	ISR	32 ms DMA	—	Hand the buffer to the DSP queue
`emg_dsp`	1	20	Queue	8 kB	Biquads, envelope, FFT, clip count
`imu_task`	1	18	10 ms timer	4 kB	Read both BMI270s, Madgwick
`analysis`	1	16	20 ms timer	6 kB	Angle, hold timers, cycle, compensation, grading
`haptic`	0	22	Queue	2 kB	Pattern playback as LEDC duty
`ble_tx`	0	12	Queue	6 kB	Frame packing and notification
`storage`	0	8	Poll	4 kB	Tier-2 writes, backfill service
`housekeeping`	0	5	1 s timer	3 kB	Battery, status, LED, watchdog

[built] **Data flow.** ADC DMA to `emg_dsp` by queue; `emg_dsp` and `imu_task` publish into a shared, mutex-free double-buffered state block that `analysis` reads at 50 Hz. `analysis` emits events into the haptic queue and the BLE queue independently — **the haptic path never waits on the BLE path**, which is what keeps feedback working during a dropout. Queue depths are four frames; an overrun sets a flag in the frame header rather than blocking the producer, because dropping a sample and recording it is always better than stalling acquisition.

[built] **Watchdog.** The task watchdog subscribes `emg_dsp`, `imu_task` and `analysis` with a 5-second timeout. A missed feed drives the device into `FAULT`: haptics muted, acquisition stopped, cause reported, LED red. The device does not silently reboot mid-set.

15 Verification

15.1 [built] What runs

Five suites run on a laptop with no hardware, no radio and no ESP-IDF. [tested] All five pass.

Suite	Covers	Result
`test/test_rep.c`	Compiles the real `rep_cycle.c`, `grading.c` and `activation.c`; drives simulated angle and EMG traces	16 scenarios pass
`test/verify_dsp.py`	Filter response and the median-frequency estimator against known signals	All pass
`host/test_protocol.py`	Wire format: every struct, every frame, every event, all twelve commands, the config TLV	All pass
`host/test_grading.py`	The Python mirror of `grading.c`	All pass
`host/test_sim_session.py`	The host client against the device simulator	All pass

15.2 [built] What the repetition suite pins

Eight scenarios became sixteen. The ones that matter most are the ones that would otherwise fail silently:

- **Ten deliberate repetitions count as exactly ten** — the 29 October acceptance gate, directly.
- **A passive swing counts nothing and starts nothing**, and reports itself (§7.2).
- **The gate needs both** muscle and movement; neither alone arms.
- **A long hold at the top counts nothing**; the count follows the return.

- **Set completion fires exactly once**, at targets of 5 and 20.
- **Tempo is measured and scored**, and a 9 s repetition scores below a 2.4 s one.
- **Smoothness counts pushes and does not reward slowness.**
- **A shoulder hike does not also register as trunk lean**, and a genuine common-mode rotation does.
- **Host configuration takes effect** — hold duration and target angle both change the outcome.
- **The activation detector does not chatter** across 100 ticks at its own boundary.

15.3 [built] Closing the simulator gap

v0.1's audit recorded a caution: the host suites test the host against `rehab_sim.py`, and the simulator was a *more capable* device than the real one — it dispatched three opcodes the firmware did not. "All tests pass" therefore did not establish that firmware and host interoperate.

That gap is closed from both ends. The firmware dispatches all twelve opcodes, and the simulator now implements the same repetition cycle, the same activation detector, the same grading and the same configuration codec as the device. Where the two could still drift — struct layouts — the layout is pinned on both sides by arithmetic over `EMG_N_CHANNELS`, with a `_Static_assert` in C and an assertion in Python.

The caution has not disappeared, and it should not. A simulator is still not a device. The first real interoperability evidence remains the bring-up gate: 60 seconds logged by the Python client with zero sequence gaps.

16 Defects fixed

The nine defects the 22 October audit found are fixed. Two more were found while fixing them, and both were wire-format bugs that only a real device would have exposed.

Defect	Resolution
`timestamp` never written	Stamped at save time from the host epoch
`is_stale(&rec, 0)`	Given the real epoch; unknown age no longer means expired
Trunk and shoulder double-count	Trunk lean is now true common mode (§6)
`rest_sum_sq` discarded	σ is stored in the record and drives the activation detector
`CLINICIAN_OVERRIDE` never set	Set by `calibration_set_record()` on a host write
Calibration step argument ignored	Honoured; any step runs on its own
Battery hard-coded to zero	Sampled while acquisition is idle
`SET_CONFIG` overloaded	Calibration writes use an internal opcode
Tier-2 capacity misstated	Stated as the 600 s the constant actually sets
Found during this work	
`device_status_t` layout mismatch	**The host expected 40 bytes with padding that the `packed` C struct does not have.** Every field after `battery_mv` — battery, sequence range, clip counters, repetition count and the clock offset — would have been read from the wrong offset on real hardware. Both sides now derive the layout from one expression, and both assert it
`features_t` layout assumed two channels	Same class of problem, found the same way. Now derived from `EMG_N_CHANNELS`, including the compiler's internal pad byte

Why the status-struct bug survived so long is the useful part. The host suites parse what the host builds, so a layout that is wrong in the same way on both sides round-trips perfectly. It took comparing `sizeof(device_status_t)` against `STATUS.size` — a C program and a Python expression, compared by hand — to see it.

That is now a build-time check rather than an act of attention: the C side carries `_Static_assert` and the Python side an assertion, both over the same arithmetic. A field added to one and forgotten on the other fails to build.

17 What remains

Everything outstanding now needs hardware, a decision, or volunteer data — there is no firmware work on the critical path for 29 October.

Needs the bench

1. **Confirm the ERM's rated voltage and overdrive clamp** against the Adafruit 1201 datasheet before the first auto-calibration. Wrong values give a weak or buzzy actuator.
2. **Scope the haptic pattern timing** on the PWM line against the 10% tolerance, with BLE active, to settle the core-0 preemption question (§9.4).
3. **Goniometer check** of the elbow angle at 0, 45, 90 and 135°, and tune Madgwick β from the result.
4. **Set the front-end gain** against the clipping counter rather than by eye, and confirm the

counter reads zero at maximum effort.

5. **Blindfolded discrimination** of all seven patterns through fabric.
6. **Two hours of battery operation**, and the low-battery behaviour at the set boundary.

Needs volunteer data

1. **Tune the grading weights and the tempo window.** Everything is host-settable now, so this is a tuning session rather than a rebuild.
2. **Tune the compensation thresholds** with volunteers staging each compensation deliberately.
3. **Check the activation thresholds** on a real limb — in particular how often `using_fallback` fires, which would mean rest captures are not yielding a usable σ.

Needs a decision

1. **Cue blanking** (PRD §5.6): suspend IMU and EMG analysis for the duration of each cue? Nothing is implemented, and nothing should be until the firmware and algorithms owners agree, because it changes scoring behaviour around every cue. The soft click makes this slightly more pressing, since it is the one cue that fires while the patient is moving.
2. **Repetition events to flash**, so they survive power loss independently of the PSRAM session buffer (PRD §5.9). The partition table reserves space; nothing writes to it.

Phase 2, with the custom PCB. Set `EMG_N_CHANNELS` to 2. That restores the triceps channel, the co-contraction component at its PRD weight, the triceps maximum as calibration step 4, and the two-channel orientation test in the donning check. Every one of those paths is written and guarded rather than deleted, and the frame layouts, the calibration record and the status struct all derive their sizes from that constant.

18 Design invariant

Unchanged from v0.1, and this revision is the evidence for it:

> Channel-to-muscle mapping, electrode sites, thresholds, hold durations and scoring weights are **configuration**, not structure. Changing the measured muscle pair or moving the electrodes touches a header file — not the acquisition, timing, transport or state machine layers.

This revision moved the actuator, halved the channel count, moved where a repetition is counted, replaced the scoring model, replaced the muscle-on detector, rebuilt the calibration protocol and added a runtime configuration layer. The acquisition path, the DSP chain, the fusion layer, the buffering tiers and the task architecture absorbed all of it without structural change.

The one place the invariant had to be *enforced* rather than relied upon is the channel count, which reaches into three wire formats. It is now a single constant that both sides derive from, with

assertions on both ends — which is what the invariant looks like when it is load-bearing rather than aspirational.